Stock Eat



Raúl Rosado Refolio

Mat. 394696

Programmazione di Sistemi Mobili – Project Memory

Università degli Studi di Parma

Index

1. **Introduction** 3
2. **Application Description** 4
3. **System Architecture** 5
4. **Technology Stack** 6
   - **Frontend** 6
   - **Backend** 6
   - **External Services** 7
5. **Data Model** 8
6. **Application Sections** 9
   - **Inventory Screen** 9
   - **Recipes Screen** 9
   - **Shopping List Screen** 10
7. **System Requirements** 11
   - **Functional Requirements** 11
   - **Non Functional Requirements** 12
8. **Testing** 13
9. **Installation and Use Guide** 14
10. **Possible Future Improvements** 15

1. Introduction

StockEat is a project born from the personal need to optimize the use of the food we buy week after week, with a multiple objective: to reduce food waste, save significant amounts of money and organize our pantries in a simple, accessible and intelligent way.

This project addresses the problem outlined above with an approach that goes beyond a simple organizational app. It's a tool that integrates Artificial Intelligence into its process, leveraging its advantages to generate recipes with clear and detailed instructions, prioritizing the most perishable ingredients.

2. Application Description

2.1 Main Functionalities

StockEat is a native Android app designed for users who want to manage their pantries efficiently and sustainably. The app integrates inventory management, artificial intelligence, a shopping list, and other features. Its main functionalities are detailed below:

Functionality	Description
Inventory Management	Viewing, editing, partial/total consumption, and deletion of stored food items. Displaying the expiration date, quantity, name, and image of each stored product.
Barcode scanning	Add products faster by scanning their barcodes with your device's camera. Data is provided by the free, public Open Food Facts API.
Manual addition of products	Product registration using a simple form, where the user can specify the name, quantity, and expiration date of the product to be added.
AI-powered recipe generation	Creation of custom recipes either from the ingredients available in the inventory, prioritizing those that are more perishable, or from a list of ingredients provided by the user at the time.
Recipe Saving	Local storage of generated recipes for later reference and/or preparation.
Shopping list	Registration of products to purchase, with automatic addition when stock runs out, or manually.
Notifications	Push notifications when a product is near its expiration date

2.2 General Usage Flow

The typical user flow in the application begins in the inventory section, after starting the application, where they can add products manually or by scanning their barcode (Consulting the Open Food Facts API in this case).

Once the inventory is populated, the user can access the ‘Recipes’ section to generate recipes by making a request to the Flask backend, which in turn will prepare a structured prompt to send to the Gemini 2.5 Flash AI model, which, based on its ingredients, will generate several recipes, which can be saved by pressing the corresponding button for later consultation in the saved recipes tab (‘Saved’).

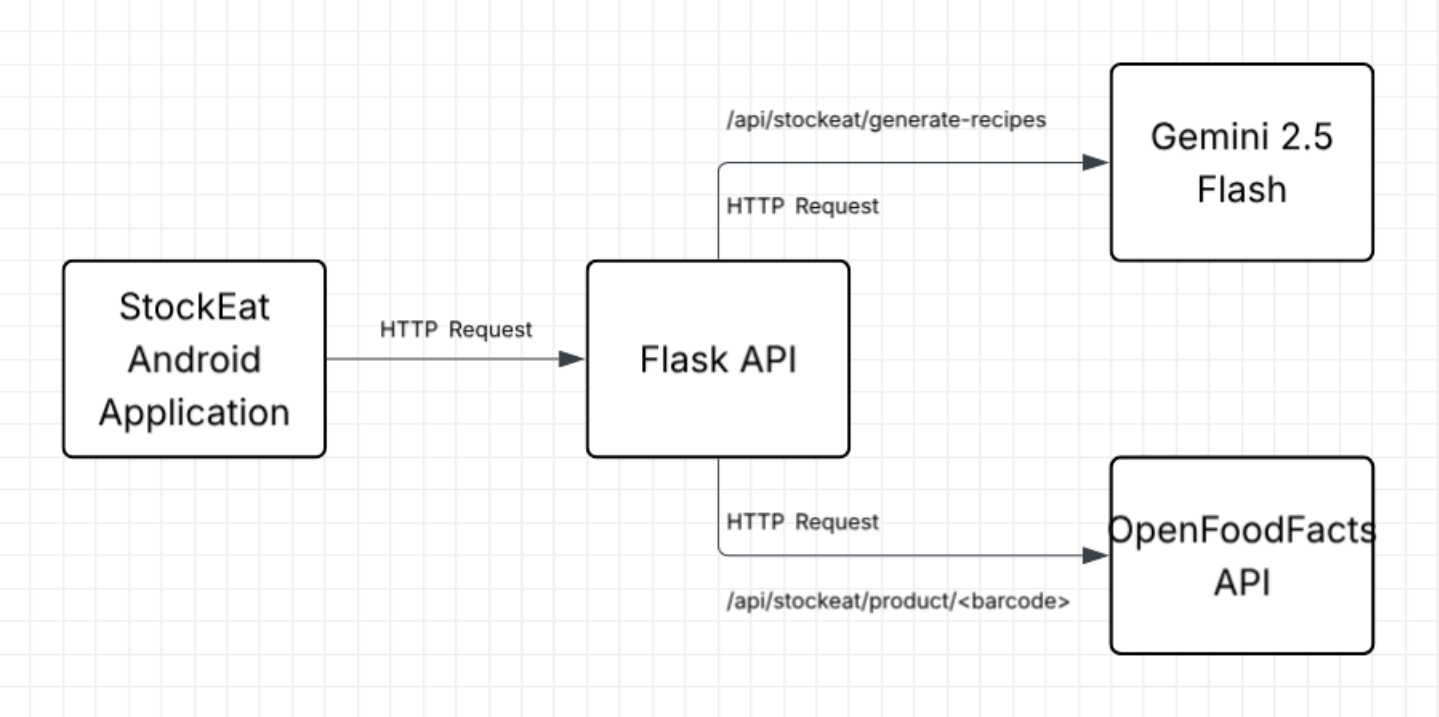
When the user decides to prepare a recipe, they can return to the 'Inventory' section to subtract the amount used of each product in their preparation, and if the remaining amount is equal to 0, a modal will be shown to the user to decide whether to add the product to their Shopping List.

Finally, the user can access the Shopping List section to complete it with the products they need to buy soon, thus completing the general flow of use of the application.

3. System Architecture

3.1 Client-Server Architecture

StockEat is based on a client-server architecture, with two distinct layers. On one hand, there's the Android client, and on the other, a simple RESTful API developed in Python with Flask, whose main function is to act as an intermediary between the application and external elements such as the Gemini model and the Open Food Facts API.

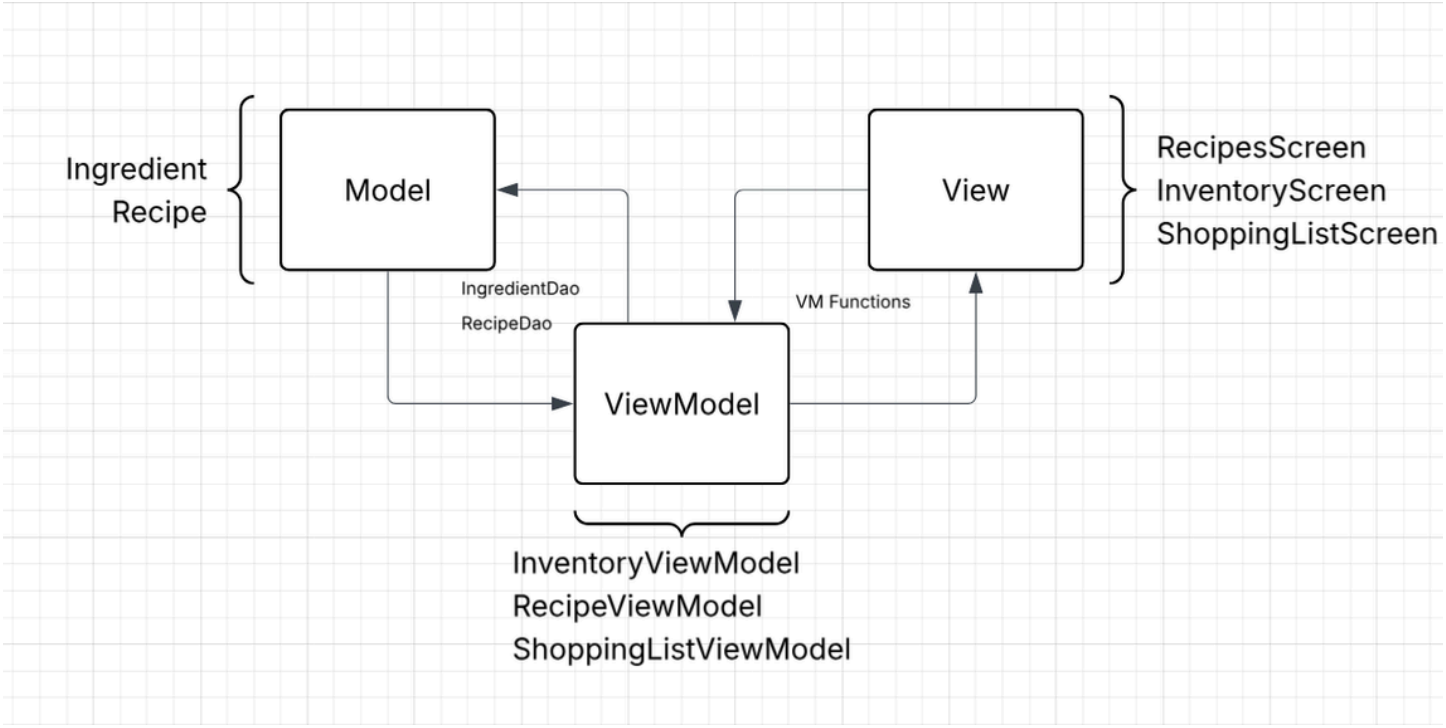


3.2 Android Internal Architecture

The Android application is internally based on the typical Model View ViewModel (MVVM) architecture, establishing a clear separation between user interface and business logic.

The three layers of this pattern are organized as follows:

- Views are composable functions in Jetpack Compose, whose purpose is to build the user interface. They are reactive and update immediately when the state exposed by the ViewModel changes. In this case, the main views are those corresponding to Inventory, Recipes, and Shopping List.
- ViewModel: These are the classes that act as intermediaries between the UI and the data. Each view mentioned above has access to its corresponding ViewModel.
- Model: data layer composed of Room entities, DAOs, etc. In the case of StockEat, these correspond to the ‘Ingredient’ and ‘Recipe’ models.



4. Technology Stack

4.1 Frontend

Library/Tool	Use in the project
Kotlin	Main language of development
Jetpack Compose	UI Construction
Room	Persistent data storage
Retrofit 2	HTTP client for consuming APIs
Moshi	JSON Converter
ML Kit / CameraX	Using the camera to scan barcodes
Coil	Image upload in the UI
WorkManager	Notifications management

4.2. Backend

Library/Tool	Use in the project
Python	Main language in backend development
Flask	Building the REST API
Render	Deployment

4.3 External Services

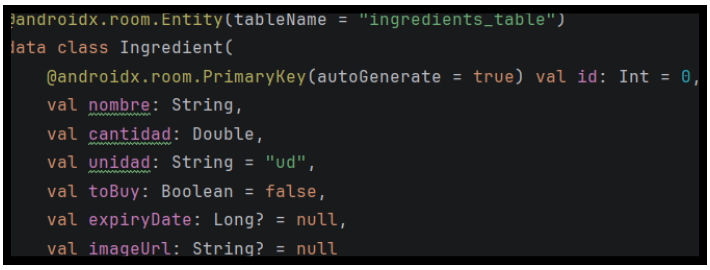
Library/Tool	Use in the project
Gemini 2.5 Flash	AI model used for recipe generation.
Open Food Facts	Free API used to query products using their barcode.

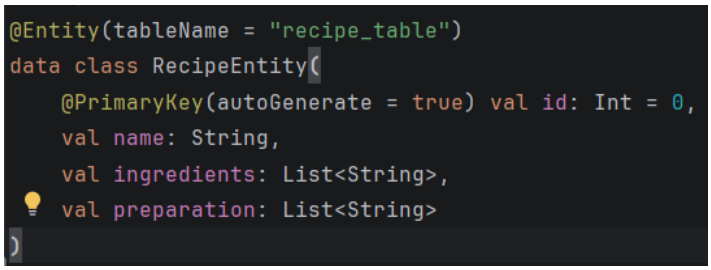
5. Data Model

5.1 Main Entities

StockEat's local persistence is managed with Room, which provides an abstraction layer over SQLite. The data model consists of the following entities:

Entity	Main Attributes
Ingredient	id (Int), name (String), quantity (Double), unit (String), toBuy (Boolean), expiryDate (Long?), imageUrl (String?)
Recipe	id (Int), name (String), ingredients (String), preparation (String)



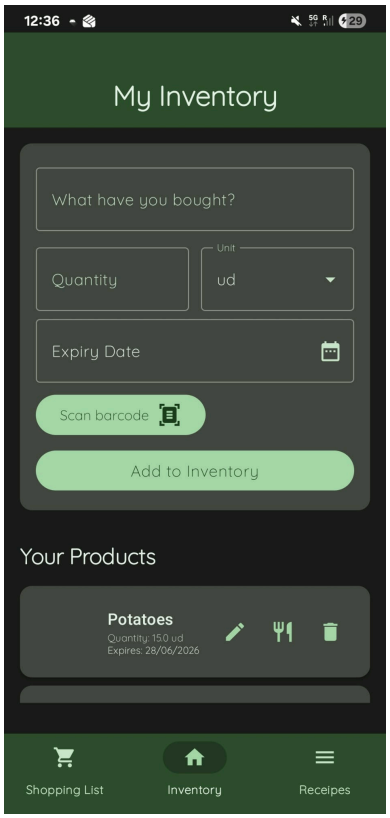


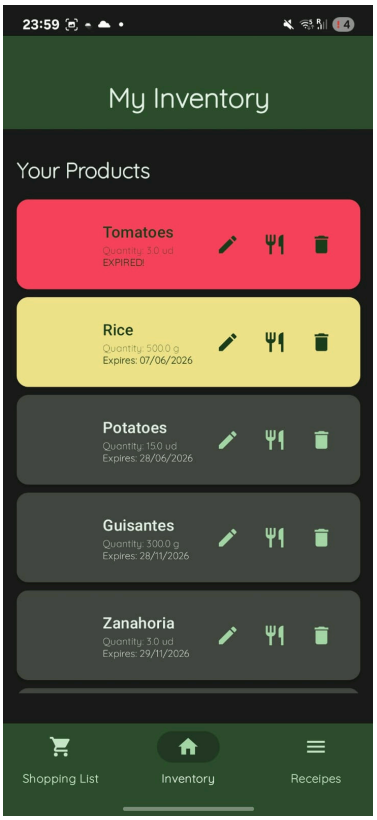
6. Application Sections

6.1 Inventory Screen

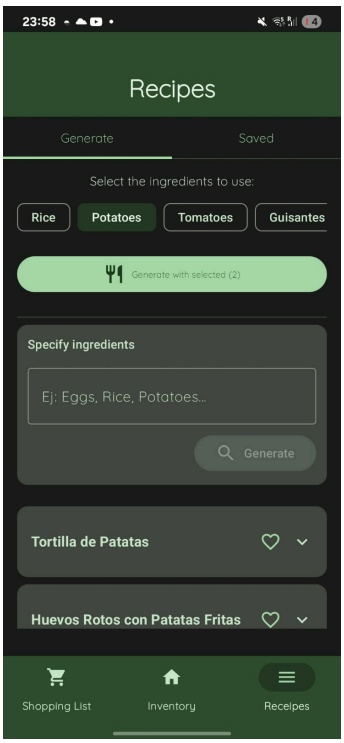
The application's main screen is the inventory. It displays a form that allows the user to add products by specifying a name, quantity, the corresponding unit of measurement, and its expiration date. It also provides a button with the option to expedite this process by scanning the product's barcode.

The user can also view all the products they own. Each inventory item displays the product name, the available quantity with its corresponding unit of measure, and the expiration date, which turns yellow if it is less than 2 days from expiring and red when that date has passed, indicating that the product has expired.





6.2 Recipes Screen



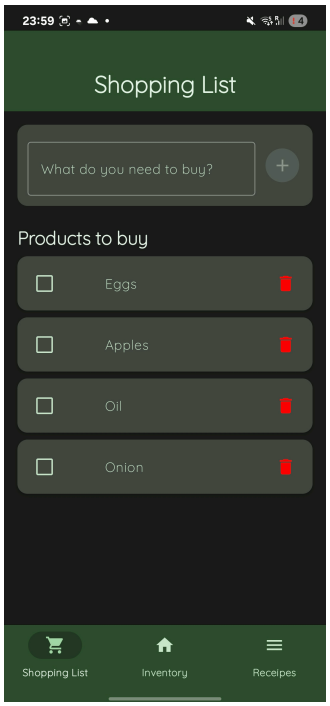
The recipes section contains a main button that allows the user to generate recipes from the selected ingredients available in the inventory. It also includes a text field where the user can manually enter ingredients of their choice to generate recipes.

Once generated, the recipes will appear listed in this section as drop-down menus that the user can interact with to view the necessary ingredients and preparation instructions. Each menu item contains a button that allows the user to save the recipe in the 'Saved' section for later reference, preventing it from being deleted when the user creates new recipes.

6.3 Shopping List Screen

The Shopping List is the third main section of the application, allowing users to add products to their shopping cart.

It's a fairly simple section that provides a short form users can fill out to manually add products to the list. It's also possible to add products to this section from the inventory section when a product's stock is zero.



7. System Requirements

7.1 Functional Requirements

ID	Name	Description
RF-01	Manual registration of products	The system should allow the user to add products by specifying their details in a simple form.
RF-02	Inventory visualization	The system must display the products to the user in a clear and orderly manner according to urgency of consumption (expiration date), also indicating in red those items that have already expired.
RF-03	Editing, deletion, and consumption	The system must allow the user to edit their products, delete them, or consume a partial amount to record its use in recipes.
RF-04	AI-powered recipe generation	The system must offer two recipe generation modes: an automatic mode that uses inventory ingredients, prioritizing those closest to expiring, and a free mode in which the user manually specifies the desired ingredients without taking into account the current inventory.
RF-05	Recipe Saving	The user should be able to save generated recipes for later reference.
RF-06	Shopping list management	The system will provide a Shopping List section, to which the user can add products manually or automatically (when a product's stock runs out).
RF-07	Complete purchased products	The system will allow the user to mark those products on the list that they have purchased, automatically returning them to the inventory.
RF-08	Product registration by scanning	The system will make it easier for the user to register products by scanning barcodes.
RF-09	Notifications	The system will notify the user when a product is less than two days from its expiration date

7.2 **Non Functional Requirements**

ID	Category	Description
RNF-01	Performance	The model's response time for generating recipes should not exceed 30 seconds under normal network conditions.
RNF-02	Usability	The user interface should be intuitive and accessible.
RNF-03	Partial offline availability	Inventory viewing and management functions must be available offline.
RNF-04	Maintainability	The code should follow SOLID principles and the MVVM architecture to facilitate future extensions.
RNF-05	Privacy	User data must be stored exclusively locally, without synchronization to external servers.

8. Testing

8.1 Features Functionalities

- **Manual Addition:** Adding items manually to the inventory and shopping list works correctly.
- **Automatic Addition:** Simulating product consumption until stock runs out successfully triggers the automatic addition of items to the shopping list.
- **Restocking:** Moving items from the shopping list back to the inventory (marking them as bought and updating their data) works perfectly.
- **Expiration Alerts:** Items correctly change color based on their expiration date: **yellow** when they expire in less than 3 days, and **red** when they are expired
- **Recipe Generation:** Tested extensively. It takes about 10 seconds on average due to the natural delay of the LLM. Note: Occasional errors occurred here, mostly caused by Flask internal server errors (Code 500), due to the limits imposed by the Gemini API on the free version. Typical error log: `google.genai.errors.ServerError: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later.', 'status': 'UNAVAILABLE'}}` This could be improved using a paid model, or a LLM locally deployed.
- **Saved Recipes:** Saving recipes works as expected.
- **Consume Product:** Tested thoroughly, including edge cases (like entering negative numbers or amounts higher than what is available). The app handled all cases correctly.
- **Barcode Scanner:** Flask backend errors (Code 500) were detected, most of them caused by the cold boot inherent to Render's free tier service. When the service is up and running, it usually doesn't give this error.
- **Push Notifications:** Fully tested and working.

8.2 UI and Navigation

- **Buttons and Forms:** Every single button and text field across all forms works correctly.
- **Navigation & Fluidity:** Section changes are fast, and screen rotation works smoothly without breaking the layout. Some elements like text written on TextFields, Ingredients Chips selected on Recipes Screen and temporally Recipes items generated by the model dissappear when screen is rotated.
- **Tablet Optimization:** The app was tested on a Samsung Galaxy Tab S6 Lite, the interface adapts perfectly to larger screens.
- **Background and Sleep Mode:** Minimizing the app or putting the device to sleep causes no unexpected behavior.

9. Instalation and Use Guide

StockEat consists of two independent components: an Android client and a Python/Flask backend deployed on Render.

9.1 Backend

The backend is publicly deployed and accessible at:

“https://stockeat.onrender.com”

No local installation is required. The service starts automatically on each request. Note that on the free Render tier, the service may take approximately 30 seconds to respond after a period of inactivity (cold start).

If you want to use the backend locally, open the subdirectory called “backend” in your favourite IDE or Terminal, configure a .env file with a valid Google AI Studio API Key (with format GEMINI_API_KEY=<YOUR_API_KEY>, and run by typing “python stockeat_api.py” on terminal.

9.2 Android Application

1. Download and unzip the provided .zip file.
2. Open the StockEat folder in Android Studio.
3. Verify that the base URL in RetrofitClient points to the Render deployment URL. Change it to “http://10.0.2.15:5000/” if you want to use the backend locally (probably a better response time in this case).
4. Run the application on an emulator or physical device using the Run button in Android Studio.

No API keys or local server configuration are required.

10. Possible future improvements

The following development paths are proposed as possible improvements and extensions for future versions of StockEat:

- Cloud synchronization: Implement user authentication and inventory synchronization on a persistent database server, allowing access from multiple devices. The most natural evolution for this application.
- Image recognition: Integrate capabilities from Gemini or some other AI model to identify food through the camera, without the need for a barcode.
- Improve the responses, maybe using a recipes generation specialized model.
- Nutritional analysis: Expand the information for each food item with nutritional data.
- Collaborative mode: Allows multiple members of a household to access the same inventory in real time.